

Debugging Playbook for Ultrasound Segmentation

Dale Kernot

September 29, 2026

Contents

1	Purpose	2
2	Debugging loop	2
2.1	Step 1: batch survey	2
2.2	Step 2: spot a pattern	2
2.3	Step 3: isolate the case	3
2.4	Step 4: hypothesise from the HTML	3
2.5	Step 5: deep dive with toggles	3
2.6	Step 6: fix and re-check	3
3	Symptom to subsystem map	3
4	Existing debug toggles	4
5	Command cheat sheet	4
6	Open improvements	5

1 Purpose

This note records a practical workflow for finding and isolating failures in the segmentation pipeline. Cases often run to completion without crashing, so visual inspection of batch HTML is the primary way to spot errors. Once a bad case is identified, a single-image run with targeted debug plots is used to test a hypothesis about where the pipeline went wrong.

Functions named without a module prefix live in `general_functions.py`. The batch entry point is `python -m usseg.main`. The single-image entry point is `data_from_image`, exercised by `tests/single_image_processing_test.py`.

2 Debugging loop

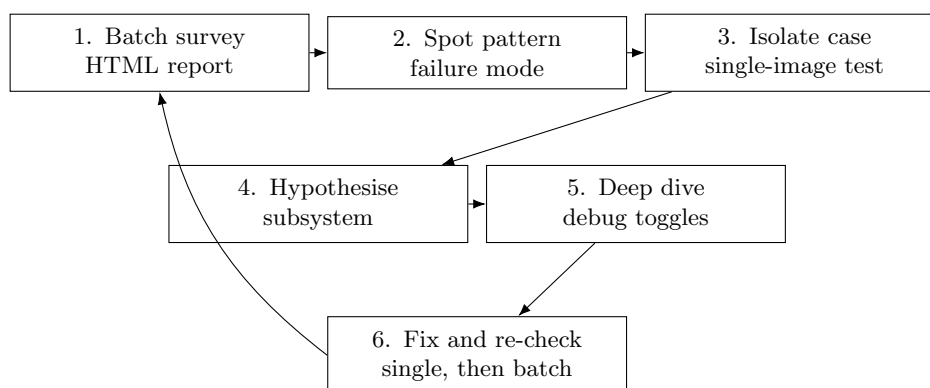


Figure 1: High-level debugging loop. Batch HTML finds patterns; single-image runs confirm the failing subsystem.

2.1 Step 1: batch survey

Configure `config.toml` (`root_dir`, `output_dir`) and run from the repository root with the project environment active:

```
python -m usseg.main
```

This finds candidate images, segments them, writes pickles and intermediate figures under `output_dir`, and produces `generated_segmented_data.html` in the working directory. Open the HTML and scan for systematic problems: wrong scale, bad envelopes, missing digitised metrics, doubled mean waves, and so on.

Note. Batch mode is deliberately quiet on debug figures. Do not rely on the `SHOW_*` toggles during a full run; they are for single-image investigation.

2.2 Step 2: spot a pattern

Ask whether the failure is isolated or shared across the dataset. Shared failures often point to a method or calibration assumption (for example right-axis ticks on a particular scanner layout). Isolated failures may be image quality, unusual annotation layout, or a single bad segmentation seed.

Record for each suspect case:

- source image path (from the HTML row / output prefix),
- which HTML columns look wrong (annotated, digitised, mean wave, table),
- which model looks best or worst (ray / morph / grow), if relevant.

2.3 Step 3: isolate the case

Copy the failing path into `tests/single_image_processing_test.py` and run:

```
python tests/single_image_processing_test.py
```

This calls `data_from_image` on one file and plots the returned trace. A clean exit does *not* mean the result is correct: the same silent inaccuracies seen in HTML can appear here. Use the batch hypothesis to decide what to inspect next.

2.4 Step 4: hypothesise from the HTML

Use the visible HTML columns as a coarse fault tree. Envelope shape is decided before beat metrics; axis calibration is decided before digitised y values make sense. Table 1 summarises common mappings.

2.5 Step 5: deep dive with toggles

Near the top of `general_functions.py` (around lines 277–285) are module flags that enable intermediate matplotlib figures:

```
SHOW_BEAT_DEBUG_SUBPLOTS = False
SHOW_GROW_DEBUG_PLOTS    = False
SHOW_MORPH_DEBUG_PLOTS   = False
SHOW_RAY_DEBUG_PLOTS     = False
SHOW_TICK_LABEL_DEBUG_PLOTS = False
```

Turn on only the flag(s) for the suspected subsystem, re-run the single-image test, and inspect the windows. Leave batch flags off again before the next full `main` run.

2.6 Step 6: fix and re-check

Confirm on the single image first, then re-run a small batch or the full `main` HTML to check that the fix did not regress other cases.

3 Symptom to subsystem map

Table 1: HTML / visual symptoms mapped to likely code areas and debug entry points.

What you see	Likely area	Start here
Axis scale wrong; digitised y systematically high/low	Tick / label finding; axis calibration	<code>search_for_ticks</code> , <code>search_for_labels</code> , <code>validate_axis_ticks</code> ; <code>SHOW_TICK_LABEL_DEBUG_PLOTS</code>

What you see	Likely area	Start here
Envelope shape wrong for one method	That segmentation method	<code>SHOW_MORPH_DEBUG_PLOTS</code> , <code>SHOW_RAY_DEBUG_PLOTS</code> , or <code>SHOW_GROW_DEBUG_PLOTS</code>
Envelope looks OK; PS/ED or feet wrong; doubled / empty mean wave	Beat / foot detection	<code>SHOW_BEAT_DEBUG_SUBPLOTS</code> ; also <code>mean_wave</code> / <code>_beat_detection_pass_feet</code>
OCR values present; digitised columns empty or all red	<code>plot_correction</code> fill, or beat detection failed for that series	Logs + beat debug; check peaks/troughs on figure 2
Crash, missing table, or blank HTML row	Earlier stage: text OCR, coarse ROI, axes	<code>batch_main.log</code> ; single-image traceback; <code>initial_segmentation</code>

4 Existing debug toggles

These flags live in `general_functions.py`. When any of them is true, `data_from_image` keeps matplotlib figures open (`_maybe_close_figures` skips `plt.close("all")`).

`SHOW_MORPH_DEBUG_PLOTS` Morphological refine stages and Method 1 envelope overlays.

`SHOW_RAY_DEBUG_PLOTS` Ray-tracing main steps (clustering, yellow exclusion, picks, smooth).

`SHOW_GROW_DEBUG_PLOTS` Region-grow main steps (seed, yellow forbid, grown mask, envelope).

`SHOW_BEAT_DEBUG_SUBPLOTS` Beat / foot overlays and derivative / search-window panels inside `plot_correction`.

`SHOW_TICK_LABEL_DEBUG_PLOTS` Axis tick and label search: thresholded ROI, right-axis tick-object filter, column-score profile, retained tick contours, and OCR label boxes matched to tick centres (`search_for_ticks` / `search_for_labels`).

Note. Leave these flags `False` for batch main runs. Enable only the subsystem under investigation for single-image debugging.

5 Command cheat sheet

Batch HTML (repo root, venv active):

```
python -m usseg.main
```

HTML only, from an existing `segmented_data.pkl`:

```
python -m usseg.visualisation_html
```

Single-image investigation:

```
# edit img_path in tests/single_image_processing_test.py
python tests/single_image_processing_test.py
```

Logs from a direct `main` run are appended to `batch_main.log`. Profiler dumps from `main`'s `prof` wrapper are written as `get_likely_us.log`, `segment.log`, and `generate_html_from_pkl.log`.

6 Open improvements

Useful next steps for this playbook / tooling (not yet implemented):

- Move `SHOW_*` flags into a `[debug]` section of `config.toml` so investigation does not require editing source.
- Make each HTML row show the full source path (and a ready-to-paste single-image command) to shorten the batch→isolate step.
- Optionally record a short case log template (path, symptom, hypothesis, toggle used, outcome) for recurring failure modes.